

CANDY DIALOGUE ENGINE

Variables

1.1.0

A large part of Candy DE's power comes from its capabilities at handling variables.

Variables can be used anywhere as substitute for data: inside spoken text, as substitutes for actor references or media files, choice menu settings, and essentially everything else.

Before you proceed, be aware that this guide will throw a lot of information and details at you. Don't worry too much about memorizing everything: the most important is the '**Basics**' section, as you will likely make use of that information often.

For the '**Advanced**' section, take a glance to see what options Candy DE offers, and refer back to this document when you need to use them until they feel familiar.

You should also look at the very end of the document, at the '**Customizing Symbols**' section: it contains important information about something you should configure early on.

This is probably the most tedious part to learn about Candy DE: it can feel like a lot of abstract information at once, but you'll probably catch on and feel at ease after a couple hours of writing dialogues. If you can get past this, other features won't give you any problems.

Basics

To use variables in Candy DE, you must first setup and learn a few symbols. Symbols act as prefixes (or sometimes as suffixes) so Candy DE can know what kind of variable it's dealing with. These Data Symbols can be configured in in [Candy_Properties.gd](#).

Variable Symbols

There are 3 base variable symbols:

```
var singleton_symbol = "£"  
var node_symbol = "$"  
var vardict_symbol = "€"
```

Each of these act as **variable references**: they tell Candy DE to look for specific variables and to fetch their values.

The **singleton_symbol** is used when you want to reference a variable located inside a singleton (or autoload or global) script, like this:

`£global_script.variable_name → £globals.player_location`

The **node_symbol** is used when you want to reference a variable located inside a node, like this:

`$node_path.variable_name → $scene/characters/player.health`

As you can see, the format is similar to GDScript for node paths. Just remember not to add quotation marks around the path.

If you ever want to reference a node only, rather than a variable inside a node, you can do that too. Just write the path, and no variable:

`$node_path → $scene/characters/player`

This can be used for commands that require being provided a node. Note that you should not add `"/root/"` at the start of the path, as Candy DE already looks under the project root. Adding `"/root/"` would cause Candy DE to look for `"/root/root/scene/characters/player"`.

Finally, the **vardict_symbol** is used to directly reference variables inside the **dialogue_variables** dictionary in `Candy_Database.gd`:

`€player_name`

This is faster than writing `'£candy.dialogue_variables["player_name"]'` for variables you use often (more on the **dialogue_variables** dictionary later).

You can even go further, and reference nested dictionaries or arrays:

`£globals.player["Stats"]["Health"]["Current_Health"]`

Again, just like GDScript.

And to top it off, variable references can be used in nested dictionary keys or array indexes. Let's imagine this example:

```
var player = {  
    "inventory": ["Health Potion", "Health Potion", "Water", "Sword"]  
}  
  
var key_1 = "inventory"  
var array_index = 2
```

`£globals.player[£globals.key_1][£globals.array_index]`

In that example, Candy DE will fetch the item at index 2 ("Water") inside the player's inventory array.

Note that referenced variables can't hold Candy DE variable references as values. For example, this will not work:

```
var variable_1 = "£globals.variable_2"
var variable_2 = 5
```

If you provide the reference '£globals.variable_1' as data, Candy DE will return the value '£globals.variable_2', not '5'.

Super Variable Symbols

The base variable symbols described above will work anywhere in Candy DE except in a very few cases: when Candy DE expects the data itself to include one or more variable references.

```
var super_singleton_symbol = "££"
var super_node_symbol = "$$"
var super_vardict_symbol = "€€"
```

This is the case, for example, with the §Set command: it requires being provided a variable, an operator and a value, like this:

Variable: "£globals.player_health"

Operator: "="

Value: 10

To clarify, in this situation, Candy DE is expected to modify the Variable by the Value provided. It's not meant to fetch the variable's value like it would normally do. So how can you still make Candy DE fetch the variable to modify from a variable reference?

The solution is to use the 'super' symbols instead:

££globals.variable_to_modify

Where the value of ££globals.variable_to_modify is the variable you want the §Set command to modify:

var variable_to_modify = "£globals.player_health"

When Candy DE encounters a 'super' symbol, it will treat it the way it normally treats the base variable symbols. In the example above, '££globals.variable_to_modify' will be correctly replaced by '£globals.player_health', and the §Set command will correctly modify it.

Other examples include the §Call, §Emit and §Await commands, who must be provided a function or a signal in the same way.

To quickly recap, in those special cases:

- The regular variable symbols won't make Candy DE replace variable references with the values of these variables. They will instead tell Candy DE which variable, function, or signal to use for the command's normal purpose.

- The 'super' symbols will actually make Candy DE replace variable references with variable values.

Don't worry about getting confused as for when to use base symbols or 'super' symbols: it's very obvious, especially once you become familiar with the commands you use. As mentioned earlier, you would use 'super' symbols when the data is expected to be or to contain variables, and very few commands are concerned.

In addition, in the Candy Dialogue Creator, data fields where the 'super' symbols should be used will have *SUPER* in their tooltips.

Role Symbol

Roles are used when you don't know in advance which actor will speak a line or be the target of a command. This is the case, for example, when you know the player's second companion should speak: instead of writing a speaker reference, you would write a role reference (e.g. "companion_2"). Then, at runtime, when an NPC becomes the player's second companion, you would update the actor reference assigned to the given role in the **roles** dictionary (**Candy_Database.gd**).

However, Candy DE needs to know when a reference is an actor, and when it is a role. This is where the **role_symbol** comes in:

```
var role_symbol = "°"
```

In the **roles** dictionary (**Candy_Database.gd**), role names must always start with with the **role_symbol** (default '°'):

°companion_2

When you write a role reference as data in a line, you must write the role name exactly (**role_symbol** included), and the symbol lets the engine know that it's dealing with a role reference, not an actor reference.

It goes without saying, actor references must never start with the **role_symbol**, otherwise the engine will mistake them for role references.

Role references can be used to fetch data in the **actors** dictionary. For example, you can type this:

°companion_2["Display Name"]

To clarify what's special about this, the **"Display Name"** key belongs to an actor in the actors dictionary, it doesn't exist in the roles dictionary. However, Candy DE will manage to lookup the display name of the actor who is assigned to the role. So if Alice is assigned to the role, Candy DE will fetch the value "Alice Turner".

Of course, the above works for any key or array index in the actors dictionary, including nested keys/arrays.

Advanced

Text Variables

You can display a variable's value in spoken text. This is done by using two symbols:

```
var text_var_symbol_start = "{}"  
var text_var_symbol_end = "}"
```

text_var_symbol_start indicates where a variable begins in spoken text (default '{}')

text_var_symbol_end indicates where a variable ends (default '}')

As an example, you would write the player's name like this:

"Hello, {}\$globals.player_name}!"

This will display as:

"Hello, Jack!"

Of course, this is a bit long to write and therefore not ideal. This is where the **variables** dictionary in **Candy_Database.gd** shows it's purpose!

Let's add add a key named **"player"** to the dictionary and assign it the player's name as a value:

```
var variables = {  
    "player": "Jack",  
}
```

In a spoken line, we can now just do this:

"Hello, {}€player}!"

This effectively cuts down on long syntax. However, it requires adding the variables to the dictionary beforehand, so you may want to use this option only for variables whose value won't change often, and which you'll often reference.

You can also use a role reference:

{ }°npc_1}

This, however, will display the actor reference assigned to the role. Instead, you'll probably want to display data of the actor. Not a problem, you can just do this:

{ }°npc_1["Display Name"]}

The example above will lookup the actor assigned to the role, and will then display the value of the actor's **"Display Name"** key inside the **actors** dictionary. It works for any key.

But wait, it gets better! You can display any other property of an actor assigned to a role (from the **actors** dictionary) by just typing the actor's reference, then the key you want to display:

`{ }Alice["Display Name"]}`

No special symbol needed this time: if there is no prefix symbol (i.e. £, \$, € or °) inside '{ }', Candy DE automatically assumes it's an actor reference.

Nested keys and arrays indexes also work, like this:

`{ }Alice["Stats"]["Intellect"]}`

`{ }Alice["Inventory"][3]`

If you don't provide any keys or arrays after the actor reference, Candy DE will display the "Short Name" key by default, if the actor has that key. Otherwise, it will display the actor's reference.

Note that this automatic defaulting to "Short Name" will only work with actor references, not role references. If you provide a role reference with no nested keys or indexes, it will display the actor reference assigned to the role.

In case you're wondering, the default symbols '{ }' and '}' won't conflict with curly brackets if you want to display them on screen:

"Hello, { }player}" → displays as "Hello, {Jack}!"

In the example above, Candy DE won't see the first '{' as the **text_var_symbol_start** because it's looking for '{ }' specifically.

Once it has found '{ }', it looks for the next '}' it finds in the line, and treats it as the **text_var_symbol_end**. The first '}' is therefore not displayed, but the second '}' is safe.

text_var_symbol_end can therefore be customized to any symbol you want, as long as it is never found inside a variable reference. So just avoid symbols that you may include in variable or key names, and also avoid quotation marks and square brackets.

text_var_symbol_start, on the other hand, must be chosen carefully: it should be something you can type quickly, as you'll probably use it a lot. But it must also be a character (or a combination of characters) that you will never type anywhere else, especially in spoken text.

More information about customizing symbols is provided at the end of this document.

Substitution Symbols

The Word Substitution feature is used to replace a word with another, which at first glance seems somewhat similar to what the **text_variable_symbols** do. However, Word Substitution is conditional on specific properties of an actor, such as gender.

For example, using the **substitution_symbol**, you can write "{}player} is a really cool guy!" and mark the word 'guy' for substitution, so that it is automatically replaced with 'girl' if the player is female.

```
var substitution_symbol = "•"
var separator_symbol = "!"
```

To achieve this, you would actually write:

"{}player} is a really cool •guy!Player•!"

Then, you would add a "guy" or "guy, girl" key to the **substitution_table** in **Candy_Database.gd**, indicate that it must be compared to an actor's **"Gender"** key in the **actors** dictionary, and provide substitute words for each possible value:

```
#^ Gender substitution dictionary:
var substitution_table := {
    "guy": [
        { "Gender": {
            "M": "guy",
            "F": "girl",
        } },
    ],
}
```

Upon encountering **•guy!Player•** in spoken text, Candy DE will look in the **substitution_table** how to evaluate 'guy', and will then check the value of the Player's **"Gender"** key as written in the **actors** dictionary. If the value is "M", Candy DE will write 'guy'. If "F", Candy DE will replace it with 'girl'.

This is all explained in more detail in the **Word Substitution** guide.

Resource & User Paths

Where commands require a reference to a file in your project's resources (such as portraits for Spoken Lines, or a media file for media commands), you can provide a resource path instead. There is no data symbol for these, just write "res://" followed by the path to the file you want to point to.

Whenever the engine encounters "res://" it will automatically know it's a resource path and treat it as such.

This also works for accessing user files, with "user://".

Note that variables can contain resource and user paths in the form of strings. For example, you can provide '£globals.file' as data to a command, where globals.file = "res://folder/image.png". Candy DE will resolve both: the variable first, and the resource path second.

The final result: the command will be provided the file 'image.png' to use.

External Variables

Candy Dialogue Engine can store and read variables in external .txt documents.

To use this feature, you can use **v_res://** and **v_user://** to designate paths to a file containing variables ('v' is short for 'variable'). You must also use the new file_var_symbol (default ↵) to separate the file path and the variable. For example:

```
v_res://path/path/file_name.txt↵external_variable  
v_user://path/path/file_name.txt↵external_dictionary["key_1"][0]
```

In places where you would use super symbols, you can write **sv_res://** and **sv_user://** ('s' is short for 'super').

Candy Dialogue Engine will automatically create the file if it doesn't exist, and add or modify variables inside of it. **This allows unlimited variable declaration at runtime.**

No option exists to delete a variable from a file at this time. It may be added in the future.

↓↓↓ **SCROLL DOWN FOR CUSTOMIZING SYMBOLS** ↓↓↓

Customizing Symbols

It's recommended that you customize symbols to fit your keyboard layout and your own preferences for quicker typing. You should do so early, before you start using them a lot in your dialogues, or you will have a lot of editing to do.

When customizing symbols, follow these rules carefully:

1. Chosen symbols must be unique from each-other

Exception: `text_var_symbol_start` and `text_var_symbol_end` can be identical.

2. Symbols can be made of multiple characters.

3. Symbols must not begin with other symbols. For example:

```
singleton_symbol = "£"  
role_symbol = "£$"      (begins with singleton_symbol)  
→ NOT OK
```

Exception: `super_singleton_symbol`, `super_node_symbol`, `super_vardict_symbol` can begin with `singleton_symbol`, `node_symbol`, `vardict_symbol`.

```
singleton_symbol = "£"  
super_singleton_symbol = "££"  
→ OK
```

4. Symbols must not contain letters or numbers.

5. ALL symbols should never appear in dictionary keys, variable names, or values (except where the purpose of these symbols requires it. For example, role references must start with the `role_symbol`).

6. ALL symbols must not contain any of the following characters:

`# \ % " ' [ ]`

7. ALL symbols must not contain any of the following characters:

`/ , . : ; _ - + * ÷ × = < > ! ? % & | ^ ~ ( ) { }`

For compatibility with future updates,
also avoid moderately common mathematical symbols,
for example: $\sqrt{\quad}$ ∞ $\pm$ $\leq$ $\geq$, etc.

Exception: `text_var_symbol_start`, `text_var_symbol_end` and `substitution_symbol`.

8. `text_var_symbol_start` and `substitution_symbol` should never appear in spoken text meant to be displayed on screen.

9. `file_var_symbol` must not contain any characters that may be found in a file path.

10. Remember to take into consideration punctuation or characters unique to languages you might translate your game to later.

These rules are necessary to avoid conflicts, otherwise Candy DE could misinterpret symbols with code syntax, operators, variable/key names or values, or with spoken text. Unfortunately, a lot of keyboard characters are already used in programming, so that limits options.

Tips:

These symbols should never cause conflicts and are safe to use:

§ ° @ ' ¬ ¢ £ \$ € ¥ • ◻ ® © ¶ ← ↓ ↑ →

For `text_var_symbol_start` and `substitution_symbol`, you can use symbols made of two or more characters to avoid conflicts with text meant to be displayed on screen. For example, if your `substitution_symbol` is '()' it won't interfere with single parentheses in dialogue texts: writing "(I wonder what to do now...)" will display exactly as written.

Consider using custom macros on your keyboard: this can save you from using ALT_GR or SHIFT+ALT_GR combinations, and can even give you access to characters not normally available on your keyboard. In addition, the Candy Dialogue Creator provides features to quickly insert '{}'' and '•|•' (or whatever custom symbols you use in place of these) in one click while writing dialogue texts.